AES Diffusion Tracing, Block Cipher Modes, and TLS Negotiation Snapshots: A Reproducible Lab Pipeline (Toy Implementation)

Course report (rd-ralph-template)

April 4, 2026

Abstract

We document an educational pipeline built around a toy AES-128 encrypt-only tracer in Python, OpenSSL-based ECB/CBC visualization on bitmap data, and OpenSSL `s_client` negotiation snapshots for a small set of public hosts. The main text includes a **primer** on bits, XOR, block ciphers, and the finite field GF(2^8) (`xtime`, MixColumns) for readers with no cryptography background, then maps each idea to `toy_aes128_trace.py`. The tracer implements the standard S-box, `xtime`-based MixColumns, ShiftRows (optional ablation), an alternate MixColumns matrix M_{new}, and key expansion, with per-step state logging for Hamming-distance diffusion curves. Results are reproducible via `scripts/run_coursework_outputs.py` and align with a homework-style research pipeline (questions → experiments → artifacts); the code is not intended for production side-channel resistance.

1 Introduction

Who this is for. We assume **no prior coursework in cryptography**. You should be comfortable with binary, hexadecimal, and the idea that files are bytes; everything else—finite fields, S-boxes, block modes, and what "AES" actually does step by step—is built up in Section 2 before we discuss the Python tracer in Section 3.

Why trace a toy cipher? In applications, encryption is often a single API call. That hides the internal *round structure* that determines how quickly a one-bit change in plaintext or key *avalanches* through the state. For learning (and for structured homework), we need **visibility**: intermediate 16-byte states after SubBytes, ShiftRows, MixColumns, and AddRoundKey, so we can plot Hamming distance along the pipeline and **see** diffusion.

What this document covers. We follow three threads that mirror a typical undergraduate lab brief and the **research-pipeline** style workflow used in `rd-ralph-template` (extract questions → design experiments → generate artifacts):

1. **AES-128 internals** (Questions 1-style): a toy encrypt-only tracer, `toy_aes128_trace.py`, with the standard S-box, GF(2^8) arithmetic for MixColumns (`xtime`), optional ShiftRows ablation, and an alternate MixColumns matrix M_{new} for comparison.

2. **Block cipher modes** (Questions 2-style): OpenSSL AES-128-ECB vs. AES-128-CBC on a BMP, illustrating why ECB leaks visual structure and CBC masks it at the byte level (Dworkin, 2001).

3. **TLS in the wild** (Questions 3-style): coarse negotiation snapshots with `openssl s_client`, to be refined with Wireshark ClientHello inventories where the assignment requires them (Dierks and Rescorla, 2008).

Reproducibility. Every figure in Section 4 can be regenerated by `python scripts/run_coursework_outputs.p` The toy AES path is checked against PyCryptodome on a single block for standard options (`verify_aes_against_pycryptodome`). This is **not** a production or side-channel-hardened implementation.

2 Preliminaries: cryptography from zero

This section explains the **minimum concepts** needed to read `toy_aes128_trace.py` and the experiments, in the order a reader typically needs them. It aligns with the **learning goals** surfaced when the homework is decomposed into questions about (i) round-wise diffusion, (ii) modes on real file data, and (iii) TLS cipher suites.

2.1 Secret writing without jargon

Encryption turns a **plaintext** message into a **ciphertext** that should look random to anyone who does not know the **key**. **Decryption** recovers the plaintext with the correct key. A **symmetric** cipher uses one shared secret key for both directions (unlike public-key systems, which we do not use here).

A **block cipher** acts on a **fixed-size chunk** of data at a time. AES-128 uses 128-bit blocks = **16 bytes**. Longer messages are handled by a **mode of operation** (Section 2.11), which repeatedly invokes the block cipher.

2.2 Bits, bytes, and XOR

A **byte** is eight bits, often written in hexadecimal (e.g. `0x01`–`0xFF`). Computers store AES state as bytes.

XOR (exclusive OR), written $\oplus$ in equations and `^` in Python for integers, combines two bits: the result is `1` if the bits differ. For bytes, XOR is applied bit by bit. Important properties: $a \oplus a = 0$, and $a \oplus b \oplus b = a$. AES uses XOR heavily (**AddRoundKey** XORs a round key into the state). If you are new to XOR, think: "flip bits where the key has a `1`."

2.3 Confusion and diffusion (intuition only)

Shannon's informal goals still guide block cipher design (Daemen and Rijmen, 2002):

- **Confusion**: the ciphertext should depend on the key in a complicated way so that changing one key bit changes the output in a hard-to-predict pattern. Nonlinear **substitution** (the S-box) provides confusion.
- **Diffusion**: changing one input bit should **spread** influence across many output bits quickly. **Permutation** (ShiftRows) and **linear mixing** (MixColumns) spread information across bytes.

Our Hamming-distance plots (Section 4) are a **quantitative** view of diffusion: after how many steps do almost all 128 bits differ between two close inputs?

2.4 Polynomials with coefficients in $\{0, 1\}$

Before finite fields, recall **polynomials** where each coefficient is only 0 or 1, and arithmetic on coefficients is mod 2 (so $1 + 1 = 0$). For example,

$$(x^3 + 1)(x^2 + x) = x^5 + x^4 + x^3 + x$$

with every coefficient reduced mod 2. This is the world of $\mathrm{GF}(2)[x]$.

2.5 The field $\mathrm{GF}(2^8)$ in one page

What problem does it solve? MixColumns must **multiply bytes** by small constants $(1, 2, 3)$ in a way that is **invertible** (so decryption exists) and **algebraically clean** (so proofs and attacks are structured). Ordinary integer multiplication mod 256 is not the right structure; AES uses a **finite field** with exactly 256 elements, written $\mathrm{GF}(2^8)$ or $\mathbb{F}_{2^8}$.

Elements = bytes. Fix an irreducible polynomial $m(x)$ of degree 8 over $\mathrm{GF}(2)$. AES standardizes

$$m(x) = x^8 + x^4 + x^3 + x + 1$$

(the "Rijndael polynomial"). Every field element is represented uniquely as a polynomial of degree < 8, hence **8 coefficients $\Rightarrow$ one byte**. The byte value is the integer whose bits are those coefficients (e.g. $x^7 + x^3 + x$ corresponds to a specific 8-bit pattern).

Addition in $\mathrm{GF}(2^8)$. Adding two field elements is **XOR of the two bytes**. This is the same as adding polynomials with coefficients mod 2. In code, every time we "XOR state with key," we are adding in the field *and* XOR-ing bits.

Multiplication in $\mathrm{GF}(2^8)$. Multiply two polynomials, then take the remainder when dividing by $m(x)$. This keeps the result inside one byte. General multiplication is implemented with bit shifts and XORs ("Russian peasant" or similar); our tracer only needs a special case.

2.6 The `xtime` operation (multiply by x)

In polynomial language, x denotes the monomial x^1. Multiplying a field element $a(x)$ by x, then reducing mod $m(x)$, is exactly what the Python function `xtime` computes on the 8-bit value a:

- If the high bit of a is `0`, left-shift by one (multiply by x with no wrap).
- If the high bit is `1`, left-shift and XOR with `0x1B` (the bit pattern of $m(x)$ minus x^8), which performs the reduction mod $m(x)$.

MixColumns builds products by 2 and 3 from `xtime`:

$$2 \cdot a := \texttt{xtime}(a), \qquad 3 \cdot a := \texttt{xtime}(a) \oplus a,$$

and uses $1 \cdot a = a$. That is why the tracer never needs a general multiply routine for the default matrix.

2.7 AES state layout

AES operates on a 4×4 array of bytes (16 bytes total), indexed in **column-major** order in our code: `s[col][row]`. A **round** applies a fixed sequence of transformations to this grid.

2.8 SubBytes and the S-box

SubBytes replaces each byte by another byte using a public lookup table, the **S-box** (`SBOX` in code). The standard table is bijective (invertible) and highly nonlinear; its algebraic structure is related to inversion in $\mathrm{GF}(2^8)$ followed by an affine map (Daemen and Rijmen, 2002). For this lab you only need: **nonlinear substitution breaks simple linear patterns** and contributes to confusion.

2.9 ShiftRows and MixColumns

ShiftRows rotates rows of the state grid so bytes move to different columns—cheap **permutation** for diffusion across columns.

MixColumns treats each column as a four-tuple of field elements and multiplies it by a fixed 4×4 matrix over $\mathrm{GF}(2^8)$. Entries are only $1, 2, 3$, implemented with XOR and `xtime`. The matrix is chosen so that changing one input byte affects all four output bytes (**branching**). Our coursework allows replacing this matrix by M_{new} (`M_NEW`) to see how diffusion curves change.

2.10 Key schedule and AddRoundKey

The 128-bit key is expanded into **eleven** 128-bit **round keys** `rk[0]..rk[10]` (`key_expansion` uses the S-box and XORs with round constants `rcon`). **AddRoundKey** XORs one round key into the entire state.

Round structure (encrypt):

- **Round 0:** AddRoundKey only.
- **Rounds 1–9:** SubBytes → ShiftRows → MixColumns → AddRoundKey.
- **Round 10:** SubBytes → ShiftRows → AddRoundKey (no MixColumns).

The tracer `encrypt_trace` follows this order and can **skip ShiftRows** or swap MixColumns to `M_NEW` for assigned ablations.

2.11 Modes (ECB vs. CBC)

The block cipher only defines $E_K(\cdot)$ on 16 bytes. To encrypt a long file, a **mode** chains blocks.

ECB encrypts each block separately. Equal plaintext blocks ⇒ equal ciphertext blocks, so **patterns survive**—visibly for uncompressed images (Dworkin, 2001).

CBC XORs each plaintext block with the **previous ciphertext block** before encrypting; the first block uses a public **initialization vector (IV)**. Identical plaintext blocks at different positions usually yield different ciphertext blocks, hiding trivial repetition.

2.12 Hamming distance on traces

We run the tracer twice with slightly different plaintext or key, logging the 16-byte state after the same sequence of steps. The **bit Hamming distance** counts differing bits between aligned states. A steep rise means **avalanche**: small input changes explode into large output differences—the empirical signature of good diffusion.

2.13 Pipeline-aligned checklist (concepts ↔ code)

Table 1 summarizes what to understand for each part of the repository.

Table 1: Concepts needed for the toy AES implementation and where they appear.

Idea	Where to look
Byte / XOR / block	`encrypt_trace`, `_add_round_key`
GF(2^8), `xtime`, $\times 2$, $\times 3$	`xtime`, `mix_single_column`
S-box / SubBytes	`SBOX`, `_sub_bytes`
ShiftRows	`_shift_rows`, option `disable_shift_rows`
MixColumns / M_{new}	`mix_single_column`, `M_NEW`, `TraceOpts.mix_matrix`
Key schedule	`key_expansion`, `sub_word`, `rot_word`, `rcon`
Diffusion metrics	`collect_hamming_curve`, `hamming_bits`

3 Methodology: `toy_aes128_trace.py`

This section maps the Python module to Section 2. Read them together: preliminaries give the **vocabulary**; here we show **which function implements which idea**. The module is **encrypt-only**, **single-block**, and for **education**; it mirrors AES-128 in FIPS-197 (NIST, 2001).

3.1 How this section follows the research pipeline

A typical `rd-agent-mcp` workflow first **extracts questions** from the homework brief, then **designs experiments** (what to measure, what to ablate), then **materializes artifacts** (JSON, PDFs). Table 2 shows how code sections answer those stages for the toy cipher.

Table 2: Homework-style pipeline stages and the supporting implementation.

Stage	Implementation / output
Clarify Q1 (diffusion, rounds)	`encrypt_trace` logging + `collect_hamming_curve` → `q1-summary.json`, Hamming PDFs
Ablate ShiftRows / MixColumns	`TraceOpts(disable_shift_rows=True)`, `TraceOpts(mix_matrix=M_NEW)`
Key vs. plaintext flip	Two runs of `collect_hamming_curve` with (P, P') or (K, K') pairs
Sanity check	`verify_aes_against_pycryptodome` in `run_coursework_outputs.py`

3.2 Constants and field arithmetic

The global `SBOX` is the AES substitution table (Section 2.8). The function `xtime(a)` is multiplication by x in GF(2^8) mod $m(x) = x^8 + x^4 + x^3 + x + 1$ (Section 2.6); every MixColumns coefficient in $\{2, 3\}$ is built from `xtime` and XOR.

3.3 MixColumns and M_{new}

`mix_single_column` multiplies one column by a 4×4 matrix over GF(2^8). The default matrix is the AES circulant with coefficients $\{2, 3, 1, 1\}$. For coursework Part C, `TraceOpts.mix_matrix` can point to `M_NEW`:

$$\begin{bmatrix} 3 & 2 & 1 & 1 \\ 1 & 3 & 2 & 1 \\ 1 & 1 & 3 & 2 \\ 2 & 1 & 1 & 3 \end{bmatrix}$$

so you can **compare diffusion curves** when linear mixing changes but ShiftRows and the rest remain.

3.4 Key expansion

`key_expansion` implements AES-128 ($N_k = 4$, $N_r = 10$). `sub_word` applies `SBOX` to four bytes; `rot_word` rotates a 32-bit word; `rcon` supplies round constants XORed into the schedule. Output: eleven round keys `rk[0..10]` (Section 2.10).

3.5 Round function and logging

`encrypt_trace(plaintext, key, opts, log)` encrypts one block. If `log` is provided, it receives `(label, round, state_bytes)` after:

- initial AddRoundKey (`r0_add_round_key`);
- for rounds 1 . . . 9: SubBytes, ShiftRows (unless `opts.disable_shift_rows`), MixColumns (using `opts.mix_matrix` if set), AddRoundKey;
- round 10: SubBytes, ShiftRows (optional skip), AddRoundKey (no MixColumns).

State layout is `s[col][row]` (column-major), matching Section 2.

3.6 Hamming analysis

`hamming_bits` / `hamming_bytes` compare two 16-byte states. `collect_hamming_curve` encrypts twice, collects two aligned lists of intermediate states, and returns per-step bit distances—the data behind the Q1 figures.

3.7 External validation

`run_coursework_outputs.py` calls `verify_aes_against_pycryptodome`: under default options, ciphertext must match PyCryptodome AES-ECB for one block. This validates functional agreement with a reference, **not** resistance to timing or power analysis.

4 Experiments

All figures below are produced by `python scripts/run_coursework_outputs.py` from the repository root (conda environment `rd-ralph-template` with Matplotlib, NumPy, PyCryptodome, Pillow; OpenSSL on `PATH`). Paths are relative to `output/diagrams/`.

4.1 Question 1: Toy AES Hamming traces

We compare all-zero AES-128 plaintext P against P' differing by a single bit in byte 0 (value `0x01`), with an all-zero key unless noted. Figure 1 shows bit Hamming distance after each traced step for the standard round function. Figure 2 disables ShiftRows (`TraceOpts(disable_shift_rows=True)`). Figure 3 uses custom MixColumns matrix `M_NEW` from `toy_aes128_trace.py`. JSON summaries, including Part D key-flip rows, are in `output/results/q1-summary.json`.

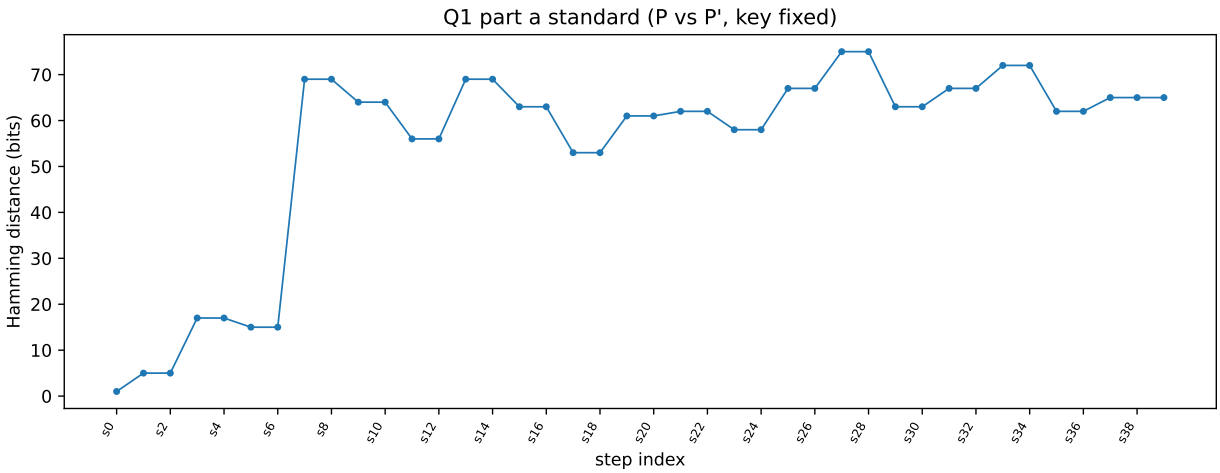

Figure 1: Bit Hamming distance vs. trace step index: baseline (ShiftRows and standard MixColumns).

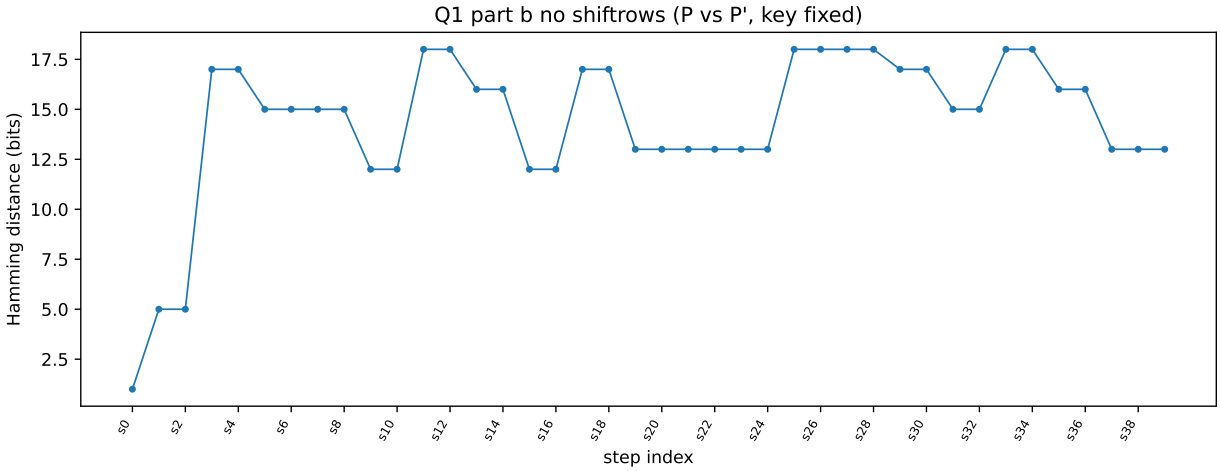

Figure 2: Same metric with ShiftRows disabled (diffusion typically slows).

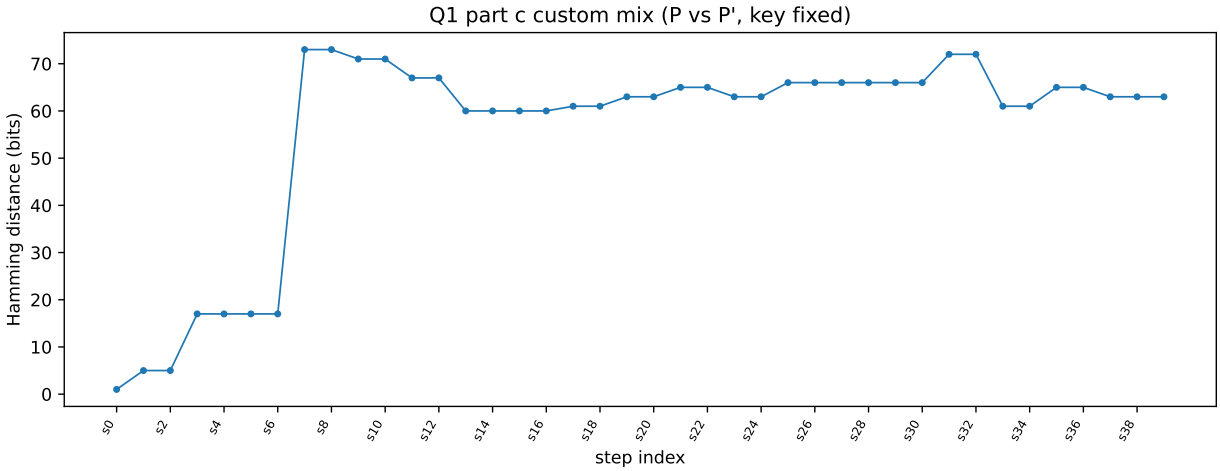

Figure 3: Same metric with alternate `M_NEW` MixColumns matrix.

4.2 Question 2: ECB vs. CBC on BMP data

OpenSSL `enc` applies AES-128-ECB and AES-128-CBC (`-nosalt`) to `assets/Secret.bmp` (a minimal gradient BMP is generated if the file is absent). Figure 4 visualizes ciphertext *bodies* as grayscale grids; Figure 5 prepends the 54-byte plaintext BMP header to ciphertext so ECB structural leakage remains visible while CBC appears more uniform at a glance (Dworkin, 2001). Key and IV hex for replay are stored in `output/results/q2/openssl_run.json` (handle per your data policy).

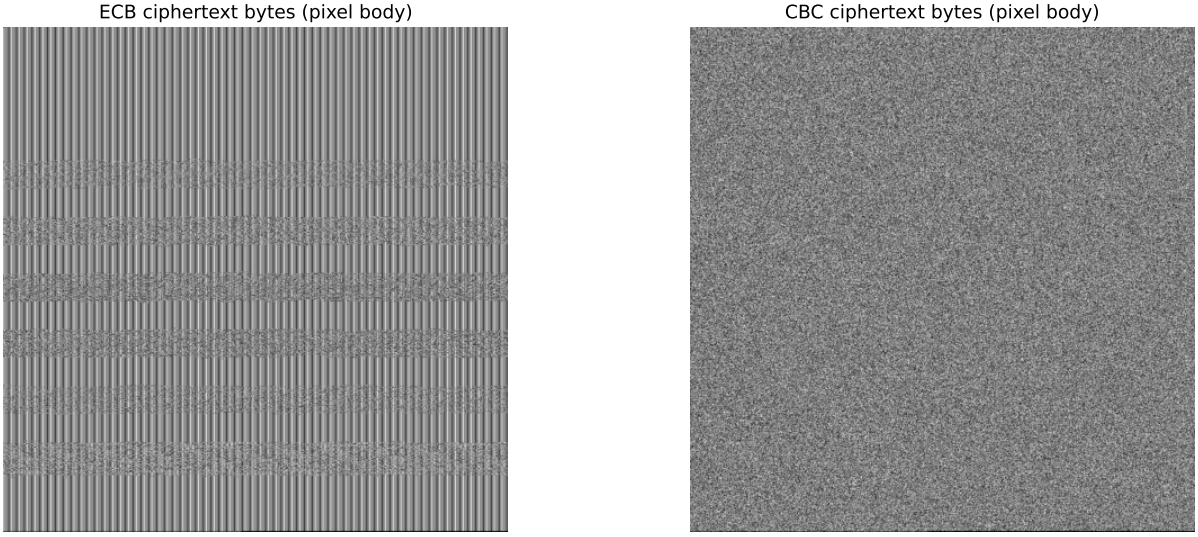

Figure 4: Ciphertext-body previews: ECB (left) vs. CBC (right).

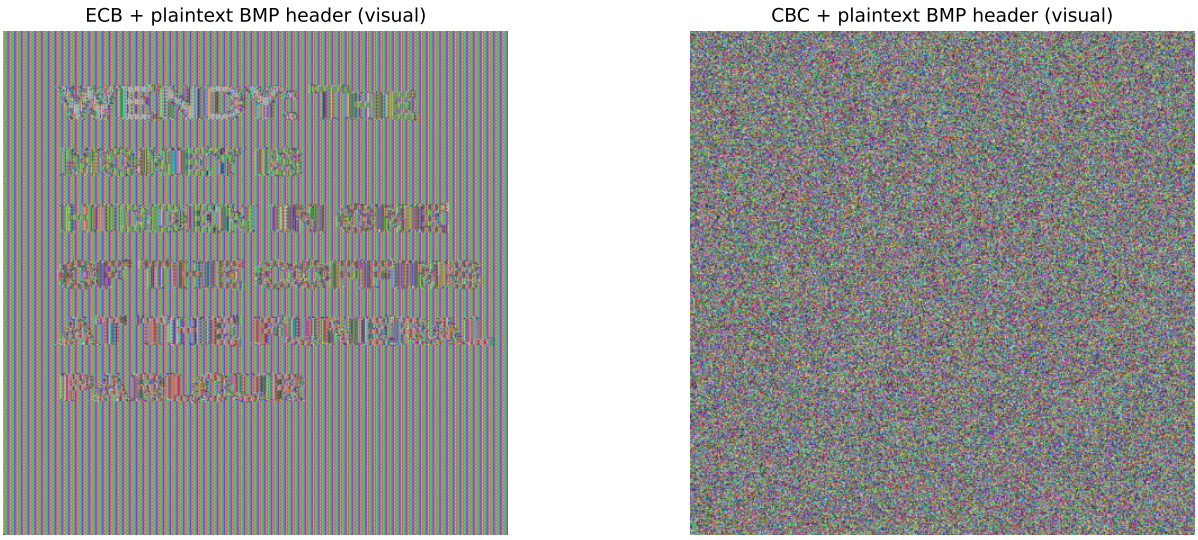

Figure 5: Plaintext BMP header spliced onto ciphertext: ECB (left) vs. CBC (right).

4.3 Question 3: TLS negotiation snapshot

`openssl s_client` connects to nine public HTTPS hosts and records negotiated protocol and cipher strings. Figure 6 tabulates the sample; Figure 7 counts cipher reuse. These plots are **not** a substitute for full Wireshark ClientHello classification required by typical coursework; they provide a reproducible PDF stand-in. Raw capture exports can be placed under `output/diagrams/q3-wireshark/`.

TLS negotiation snapshot (openssl s_client) — replace with Wireshark ClientHello tables for coursework

Host	Protocol	Negotiated cipher
google.com	TLSv1.3	AEAD-CHACHA20-POLY1305-SHA256
github.com	TLSv1.3	AEAD-CHACHA20-POLY1305-SHA256
cloudflare.com	TLSv1.3	AEAD-CHACHA20-POLY1305-SHA256
apple.com	TLSv1.3	AEAD-CHACHA20-POLY1305-SHA256
microsoft.com	TLSv1.3	AEAD-AES256-GCM-SHA384
mozilla.org	TLSv1.3	AEAD-CHACHA20-POLY1305-SHA256
wikipedia.org	TLSv1.3	AEAD-CHACHA20-POLY1305-SHA256
eff.org	TLSv1.2	ECDHE-ECDSA-AES256-GCM-SHA384
amazon.com	TLSv1.3	AEAD-AES128-GCM-SHA256

Figure 6: Negotiated parameters from `s_client` (openssl snapshot, not full ClientHello offer lists).

4.4 Template example metrics (Python demo)

The following figure is **not** from the cryptography coursework driver; it is produced by the template examples `examples/python/generate_results.py` and `examples/python/plot_diagram.py`, writing `output/results.json` and `output/diagrams/performance.pdf` (raster: `performance.png`). Include it when the final document should reflect *all* files under `output/diagrams/`.

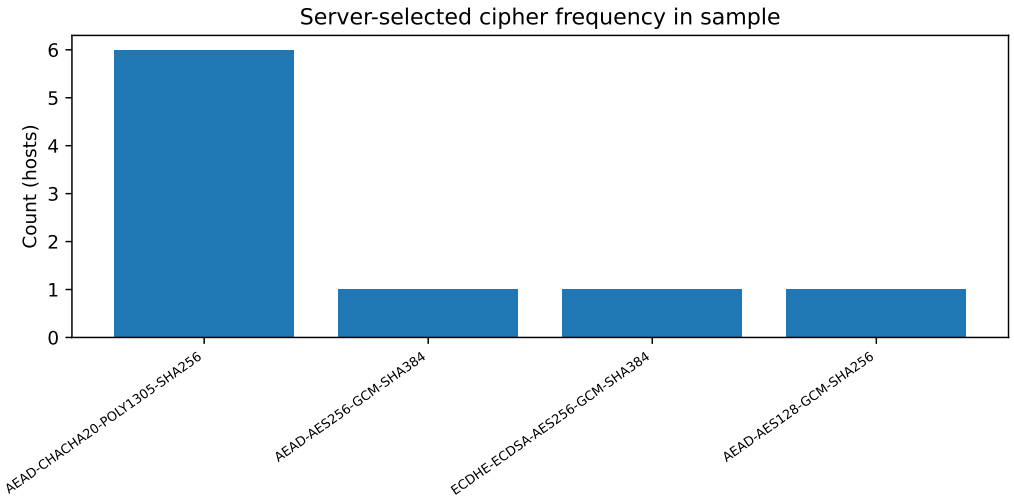

Figure 7: Frequency of server-selected cipher strings in the nine-host sample.

5 Related Work

The Advanced Encryption Standard (AES) and Rijndael's design rationale are treated comprehensively by Daemen and Rijmen (2002); FIPS-197 specifies the standard (NIST, 2001). Modes of operation including CBC and ECB are catalogued by NIST SP 800-38A (Dworkin, 2001). TLS 1.2 framing is standardized in RFC 5246 (Dierks and Rescorla, 2008). Appendix A summarizes implementation constants tied to our codebase.

6 Conclusion

We presented a unified, script-driven lab narrative for AES diffusion tracing (S-box, ShiftRows, MixColumns with optional M_{new}, key schedule, and logged states), for visual comparison of ECB and CBC on structured bitmap data, and for coarse TLS negotiation sampling. The toy AES implementation in `toy_aes128_trace.py` is intentionally narrow—single-block, encrypt-only, validated against PyCryptodome for standard options—but it makes round-wise avalanche analysis inspectable. Future extensions include multi-plaintext Hamming statistics, full Wireshark-based cipher-offer tables for two browsers and nine sites, and hardening guidance distinguishing pedagogy from production cryptography.

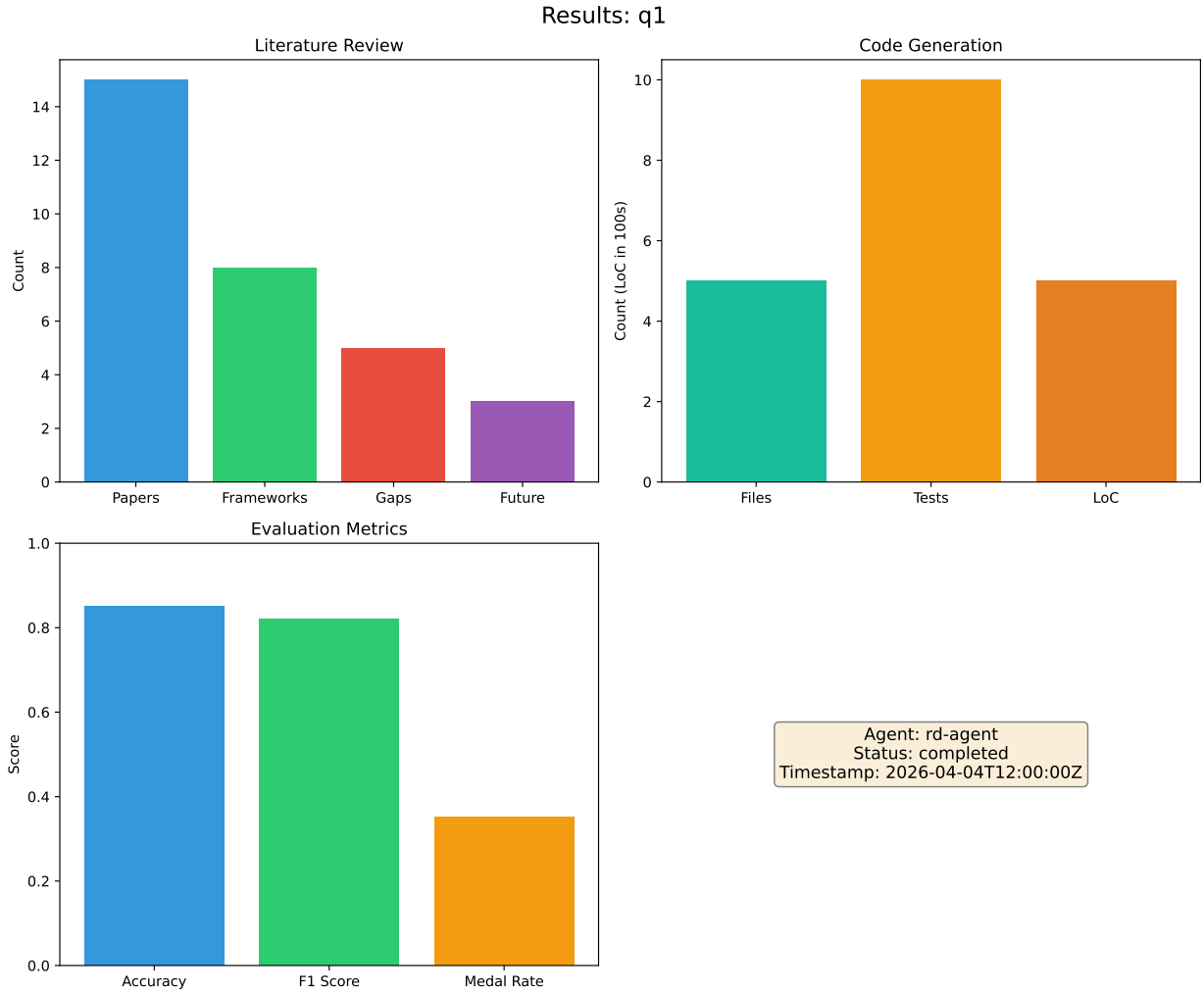

Figure 8: Illustrative metrics panels from the demo `generate_results/plot_diagram` pipeline (synthetic scores).

Reproducibility Statement

- **Code:** `scripts/toy_aes128_trace.py` (AES tracer, S-box, `xtime`, MixColumns including `M_NEW`, key expansion, `encrypt_trace`, Hamming helpers).
- **Driver:** `scripts/run_coursework_outputs.py` regenerates `output/results/*.json` and `output/diagrams/*.pdf`; requires Python 3 with `numpy`, `matplotlib`, `pycryptodome`, `Pillow`, and `openssl` on `PATH`. Q3 performs live TLS handshakes.
- **LaTeX:** Build from `src/` with `pdflatex main`, `bibtex main`, then `pdflatex main` twice. Bibliography source is `main.bib`; processed file `main.bbl` is produced by BibTeX.
- **Environment:** `conda activate rd-ralph-template` matches template defaults.

Ethics Statement

This work uses only educational cryptography artifacts and public TLS endpoints. The toy AES implementation must not be presented as side-channel resistant or standards compliant for production. ECB/CBC demonstrations intentionally highlight leakage patterns; readers should not infer that header-spliced BMP previews are security mechanisms. OpenSSL key/IV material emitted for reproducibility may be sensitive if committed to public repositories—redact or exclude per institutional policy. TLS probing should respect site terms and local law; the bundled host list is illustrative and may be replaced.

References

Joan Daemen and Vincent Rijmen. *The Design of Rijndael: AES—The Advanced Encryption Standard*. Springer, 2002.

T. Dierks and E. Rescorla. The transport layer security (TLS) protocol version 1.2. RFC 5246, 2008.

Morris Dworkin. Recommendation for block cipher modes of operation: Methods and techniques. Special Publication 800-38A, National Institute of Standards and Technology, 2001. Revision 2007.

NIST. Announcing the advanced encryption standard (AES). Federal Information Processing Standards Publication 197, National Institute of Standards and Technology, 2001.

A Implementation constants and trace labels

S-box. The byte array `SBOX` in `toy_aes128_trace.py` matches the AES specification; SubBytes applies it to each state byte.

`xtime` and MixColumns. `xtime` realizes multiplication by x in GF(2^8) with AES modulus. Default MixColumns uses coefficients 2 and 3 via `xtime` and XOR. Generic coefficient multiplication uses a double-and-add style loop for other integers.

Alternate matrix M_{new}. Coursework Part C uses `M_NEW`:

$$M_{\text{new}} = \begin{pmatrix} 3 & 2 & 1 & 1 \\ 1 & 3 & 2 & 1 \\ 1 & 1 & 3 & 2 \\ 2 & 1 & 1 & 3 \end{pmatrix}$$

(row/column convention matches `mix_single_column` in code).

Logged step order. For each round, `encrypt_trace` emits states after SubBytes, ShiftRows (if enabled), MixColumns (rounds $1 \ldots 9$ only), and AddRoundKey, enabling paired Hamming curves from `collect_hamming_curve`.

B LLM usage (writing assistance)

If applicable: LLMs may be used only for language editing; core technical content derives from the cited standards and the referenced repository scripts.